

PW1: Object-relational databases

Design and implementation of an object-relational database

During this practical work, you should demonstrate the ability to design and implement an object-relational database according to the domain description.

General rules (IMPORTANT!):

1. This task must be completed in a group. **Submission will only be available to registered groups.** A group can consist of 2 to 3 students, one of whom will be the administrator. If a student chooses to complete the task individually, they will be the sole member of their group and also its administrator. **Groups must be registered according to the procedure published on ORTUS.**
2. You should perform this practical work by using **Live SQL**.
3. During this practical work you must perform all tasks and prepare the necessary files based on the provided templates, samples, your individual variant (domain description) and task description.
4. The workload should be equally divided among all group members.
5. If a group is entitled to an emergency extension and wishes to utilize it, it is the responsibility of the group administrator to inform the instructor by email no later than the emergency extension request deadline published on ORTUS.
6. **Consultations with other groups are not allowed. Any similarities between group submissions will be interpreted as a breach of academic integrity and will result in the cancellation of the submissions and 0 points for all groups involved.**
7. **The use of any AI tools is prohibited in the development of this work.**

About this practical work:

During the generation of the PW1 variant, you will be provided with a domain description of 300-350 words. It contains the main entity sets (or object types) that must be stored, as well as the main functions that the system must perform.

Please note that the domain description is not intended to be precise user requirements – instead, consider it a general definition of the database to which you can add freely, create formulas for vague parts, etc.

The purpose of the vagueness of the domain description is twofold: (1) you may adapt the complexity of the domain according to your study goals, and (2) to prevent situations when two very similar solutions are submitted. With these domain descriptions, it is almost impossible to create two identical works unless some illegal communication has taken place (such as two groups fully working together).

Note that you do not have to implement every feature in the domain description! Read the grading rules carefully!

Your tasks:

1. Generate your PW1 variant (can and must be done by group administrator only).

In this work each group will have its own individual variant – domain description.

To get your variant please open the test “PW1: Variant generator”. Start the test, and you will see a download link. You need to download this file – it contains your group’s domain description. Then you can mark any answer and finish the test. Please note that this is not a real test but to provide you with your individual variant.

Result in your report (Table: General information):

- The name of domain you got using the “PW1: Variant generator”.

2. Develop a class diagram.

Based on your domain description, create a class diagram, which will serve as the basis for developing an extended ER diagram in the next step.

Each class should include all necessary attributes, their data types, and the methods to be implemented. Logical relationships between classes should be designed as needed.

While creating the class diagram, keep in mind that in the end (in the ER diagram), you will need **at least the following tables**:

- Table with row-type objects.
- Table with column-type objects.
- Table with object collection.

Please note that you will also need to **demonstrate the successful use of the following constructions:**

- REF columns – at least 1
- MEMBER METHODS (functions, procedures)
- MAP or ORDER FUNCTIONS – at least 1
- MEMBER PROCEDURES – at least 1
- STATIC FUNCTIONS and PROCEDURES – at least 1
- Type inheritance (including method overriding)
- Functions returning SYS_REFCURSOR

The text in diagrams must be approximately the same size as the main text in the document.

Chart drawing tools (e.g., <https://www.drawio.com/>) or CASE tools must be used to create this diagram.

Result in your report (Section 2):

- Class diagram.
- Explanations.

3. Develop the extended ER diagram.

Based on the developed class diagram, create an extended ER diagram according to the provided example (following the notation shown in the example).

If, at this stage, you realize that it is not possible to create all necessary data storage structures based on the class diagram, go back to the previous step, make changes to the class diagram, and then proceed again with this step.

The text in diagrams must be approximately the same size as the main text in the document.

Result in your report (Section 3):

- Extended ER diagram.
- Explanations (if necessary).

4. Implement and test your design (in LiveSQL).

Complete this step based on the extended ER diagram developed in the previous step.

First, decide (and note down in writing) the order in which you will create types, tables, functions, procedures, and other objects. Begin with the objects that are independent of others and proceed with those that depend only on already created objects.

In this step, please consider the following:

- Begin by creating types and tables.
- After creating each table, immediately input data (2-3 records per table, or more if you need additional data for demonstrating specific details). **Data entry must be performed only within PL/SQL blocks! Do not execute data entry using multiple separate INSERT statements!**
- After entering data in each table, immediately demonstrate data retrieval.
- After creating **all** types and tables, proceed to define type bodies (which, of course, include methods).
- After defining each type body, demonstrate the successful execution of each method within the body by writing and executing the appropriate script.
- If you implement additional functions, procedures, or triggers not included in the structures described above, implement them at the appropriate time based on their purpose, remembering to demonstrate their successful execution right away.

Write comments that briefly explain the purpose of each code section.

All comments in the code file must be included within SQL statements or PL/SQL blocks. Comments should not be outside the blocks! Please refer to the provided examples!

If you realize at this stage that modifications to the extended ER diagram are needed, return to the previous step, make the necessary changes in the ER diagram and in the class diagram, and then proceed with this step.

Prepare LiveSQL sharable link:

- **The LiveSQL script (link) must show the complete results of code execution.**

Result in your code file:

- Code (with comments) in the order required for error-free execution.
- **The code file must be obtained from the LiveSQL final script (download it after finishing your work).**
- **The LiveSQL script must fully correspond to the code file!**

5. Prepare the report!

All **diagrams** must have a **white background with black text**. Use additional colors according to the provided example.

Pay special attention to **Section 1 (Project Passport)** – prepare it carefully by thoroughly reviewing the LiveSQL script!

In **Section 4 (Group Work Report)**, you must answer the following questions:

- How was the practical work developed?
- How did you split the work? Specify contribution of each group member!
- What was productive, and what was not?

- What problems, if any, arouse and why?

The only thing missing in the report at this step will be the link to the video presentation.

6. Prepare video presentation!

Requirements regarding to the video presentation:

- All group members who participated in the project must be **visible (in video mode)** in the video presentation, and each student appearing in the video must **also speak**.
- The video presentation can be prepared as a recorded Teams or Zoom meeting with the entire group. However, groups may choose other tools for creating this video presentation.
- Presentation duration: **5 to 7 minutes**.
- During the presentation, the group should explain what each student contributed and highlight the most interesting aspects of the project that they want to draw the listener's attention to.
- The recorded video should be saved in a way chosen by the students (for example, in an RTU OneDrive account, YouTube, Google Drive), and a link to the video must be included in the report.
- **Opening the link must display an online video. Downloading must not be required!**
- The video and audio quality must be **consistently high**.
- The link must be accessible **at all times**, either **publicly (to everyone)** or **at least to anyone with an RTU account** (if stored in RTU OneDrive). **The user must not be required to specifically request access.**

Result in your report (Table: General information):

- Link to your video (e.g., YouTube)

7. Submit the report and the code file!

You must submit 2 files (on ORTUS):

- report – 1 PDF file;
- the code file as 1 TXT file (obtained from the shared LiveSQL final script)